# Memoria Práctica 2: Algoritmos Codiciosos

Algoritmia y Estructuras de Datos Avanzadas
Año Académico 2026-27

Grupo 1261 Santiago Pita y Unai Galloso

Noviembre 2026

## 1 Problema del cambio de monedas

### 1.1 Análisis experimental

En esta sección se analiza el comportamiento de la solución codiciosa (`greedy_change`) frente a la solución óptima (`optimal_change`) para cantidades comprendidas entre 1 y 500, utilizando diferentes sistemas de monedas.

#### Clasificación de las soluciones

| Sistema de monedas | Optimal (Núm / %) | Suboptimal (Núm / %) | Failed (Núm / %) | Impossible (Núm / %) |
|---|---|---|---|---|
| $[200, 100, 50, 20, 10, 5, 2, 1]$ | 500 (100.00%) | 0 (0.00%) | 0 (0.00%) | 0 (0.00%) |
| $[10, 5, 2, 1]$ | 500 (100.00%) | 0 (0.00%) | 0 (0.00%) | 0 (0.00%) |
| $[25, 10, 4, 1]$ | 500 (100.00%) | 0 (0.00%) | 0 (0.00%) | 0 (0.00%) |
| $[4, 3, 1]$ | 376 (75.20%) | 124 (24.80%) | 0 (0.00%) | 0 (0.00%) |

Cuadro 1: Resultados empíricos para $N = 500$ cantidades.

#### Tiempos de ejecución y compromiso eficiencia-calidad

| Sistema monetario ($N = 500$) | Tiempo `greedy_change` (s) | Tiempo `optimal_change` (s) |
|---|---|---|
| $[200, 100, 50, 20, 10, 5, 2, 1]$ | 0.000432 s | 0.476417 s |
| $[10, 5, 2, 1]$ | 0.000710 s | 0.366375 s |
| $[25, 10, 4, 1]$ | 0.000521 s | 0.309212 s |
| $[4, 3, 1]$ | 0.001636 s | 0.361225 s |

Cuadro 2: Comparativa de tiempos de ejecución acumulados para cantidades de 1 a 500.

**Discusión:** Como reflejan los datos de la Tabla 2, el algoritmo codicioso es entre 100 y 1000 veces más rápido que la programación dinámica (unos $\sim 0.0004 - 0.0016$ segundos frente a $\sim 0.31 - 0.48$ segundos). Esto se debe a que el algoritmo codicioso evita la exploración recursiva de subproblemas y la ordenación de listas candidatas en memoria. Sin embargo, este aumento drástico en eficiencia conlleva un compromiso en la calidad de la solución visible en la Tabla 1, en sistemas no canónicos como $[4, 3, 1]$, la solución codiciosa proporciona resultados subóptimos en el $24.80$% de los casos (124 cantidades). Por otro lado, dado que todos los sistemas evaluados incluyen la moneda de valor $1$, siempre es posible alcanzar una solución exacta, por lo que los casos clasificados como `greedy_failed` e `impossible` se mantienen en $0$ ($0.00$%).

## 1.2 Cuestiones

1. **¿Cuál es el criterio codicioso utilizado?**
   El criterio consiste en seleccionar siempre la moneda de mayor valor nominal que sea menor o igual a la cantidad restante que falta por completar en ese momento.

2. **¿Funciona correctamente para todos los sistemas monetarios?**
   No siempre. Funciona de manera óptima para sistemas monetarios *canónicos* (como las monedas de uso corriente en casi todos los países). Sin embargo, falla en sistemas arbitrarios. Por ejemplo, en el sistema $[4, 3, 1]$ para devolver $6$, el algoritmo codicioso escogerá $[4, 1, 1]$ (3 monedas), mientras que la solución óptima es $[3, 3]$ (2 monedas).

3. **¿Cuál es la complejidad temporal de la solución codiciosa?**
   Si se asume que las monedas no están ordenadas previamente, la complejidad temporal será $\mathcal{O}(m \log m)$ para ordenarlas, más $\mathcal{O}(m)$ para iterar por ellas calculando los cocientes, donde $m$ es el número de monedas disponibles en el sistema (globalmente es $\mathcal{O}(m \log m)$). Si ya estuvieran ordenadas, sería $\mathcal{O}(m)$.

4. **¿Cuál es la complejidad temporal y espacial de la programación dinámica?**
   La complejidad temporal es $\mathcal{O}(C \times m)$, donde $C$ es la cantidad objetivo a alcanzar (`amount`) y $m$ el número de monedas diferentes. La complejidad espacial es $\mathcal{O}(C)$ para almacenar la tabla unidimensional con las soluciones de los subproblemas para cada cantidad desde $0$ hasta $C$.

5. **¿Qué ventajas presenta la solución codiciosa aunque no siempre garantice el óptimo?**
   Presenta una complejidad espacial prácticamente nula $\mathcal{O}(1)$ (descontando el espacio del resultado) frente al $\mathcal{O}(C)$ de la solución exacta, y una complejidad temporal infinitamente menor que no depende de la magnitud de la cantidad $C$, por lo que es la única opción viable si $C$ es un número extremadamente grande en un sistema monetario estándar.

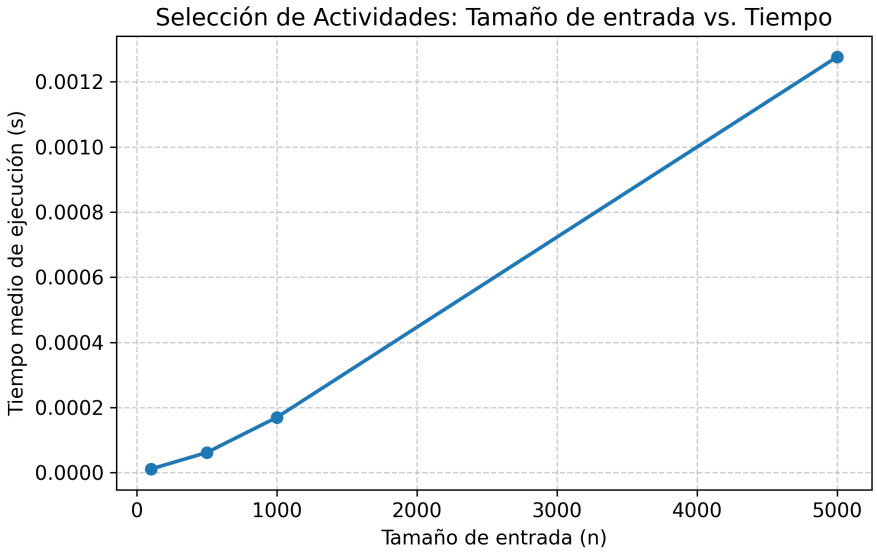
# 2 Selección de actividades

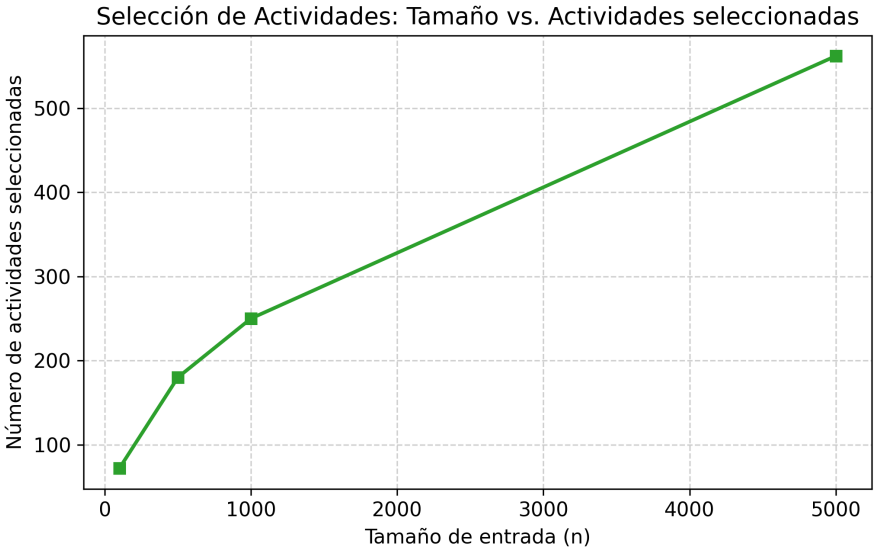
## 2.1 Análisis experimental

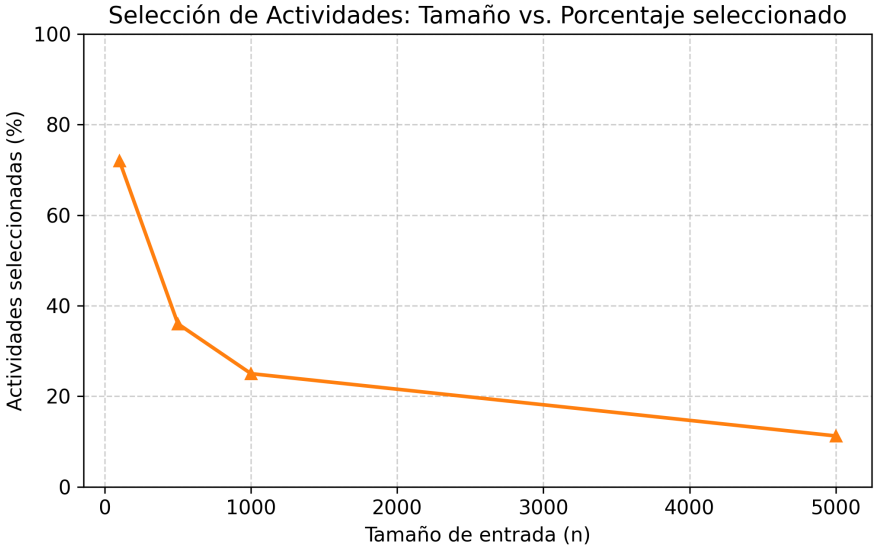
A continuación se presentan los resultados empíricos de la ejecución de `activity_selection` incrementando el tamaño del conjunto de entrada ($n \in \{100, 500, 1000, 5000\}$), promediando 10 repeticiones por cada tamaño:

| **Tamaño ($n$)** | **Tiempo medio (s)** | **Actividades seleccionadas** | **Porcentaje seleccionado (%)** |
|---|---|---|---|
| 100 | 0.000011 s | 72 | 72.00% |
| 500 | 0.000062 s | 180 | 36.00% |
| 1000 | 0.000170 s | 250 | 25.00% |
| 5000 | 0.001277 s | 562 | 11.24% |

Cuadro 3: Resultados experimentales de `activity_selection` según el tamaño de entrada.






## 2.2 Cuestiones

1. **¿Cuál es el criterio codicioso utilizado?**
   El criterio es ordenar las actividades por su tiempo de finalización ascendente (eligiendo siempre la que termine antes) y, en caso de empate, por inicio y luego identificador.

2. **¿Por qué seleccionar la actividad que termina antes deja más posibilidades?**
   Porque al finalizar lo antes posible, se maximiza el tiempo restante libre en la sala o recurso, dejando un abanico más amplio de posibilidades para que un mayor número de actividades posteriores puedan ser encajadas sin solapamientos.

3. **¿Qué podría suceder si se eligiese primero la actividad que comienza antes?**
   Que se podría elegir una actividad que comience muy temprano pero que dure prácticamente todo el día (todo el rango de tiempo de simulación), impidiendo la eje-

cución de múltiples actividades más cortas compatibles entre sí. Por tanto, no se garantizaría el máximo número de actividades.

4. **¿Qué podría suceder si se eligiese primero la actividad de menor duración?**
   También podría resultar subóptimo. Una actividad muy corta podría situarse justo en el centro de los horarios, solapándose simultáneamente con el final de una actividad anterior y el principio de una actividad posterior, bloqueando la inclusión de dos actividades a cambio de una sola.

5. **¿Cuál es la complejidad temporal y espacial?**
   La complejidad temporal está dominada por la ordenación inicial de las actividades, es decir, $\mathcal{O}(n \log n)$, donde $n$ es el número de actividades. El posterior recorrido lineal cuesta $\mathcal{O}(n)$. Por lo tanto, la complejidad temporal es $\mathcal{O}(n \log n)$. La complejidad espacial es $\mathcal{O}(n)$ para almacenar la lista de actividades ordenadas y/o la lista resultante.

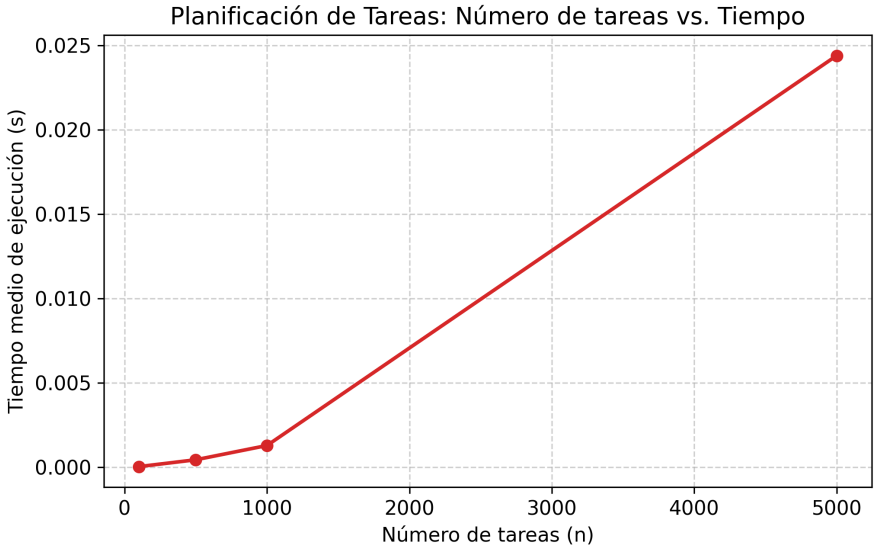
# 3 Planificación de tareas con fechas límite

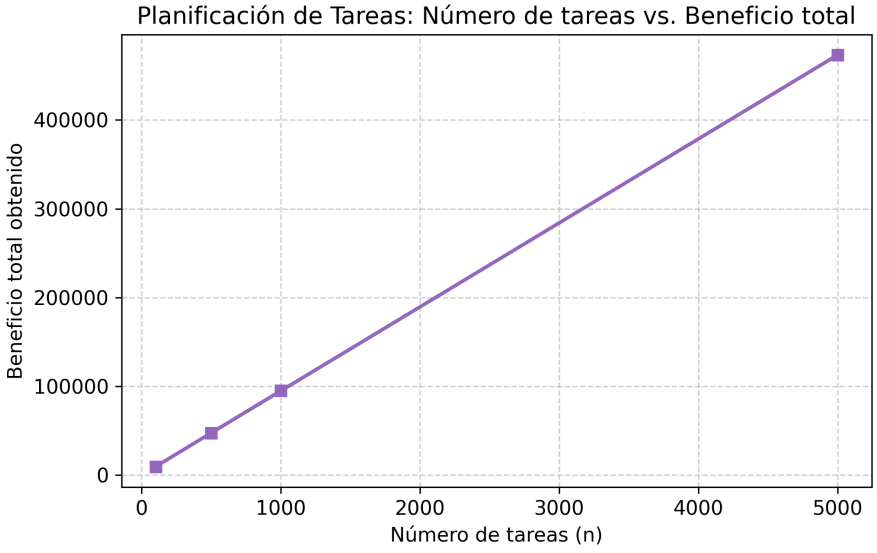
## 3.1 Análisis experimental

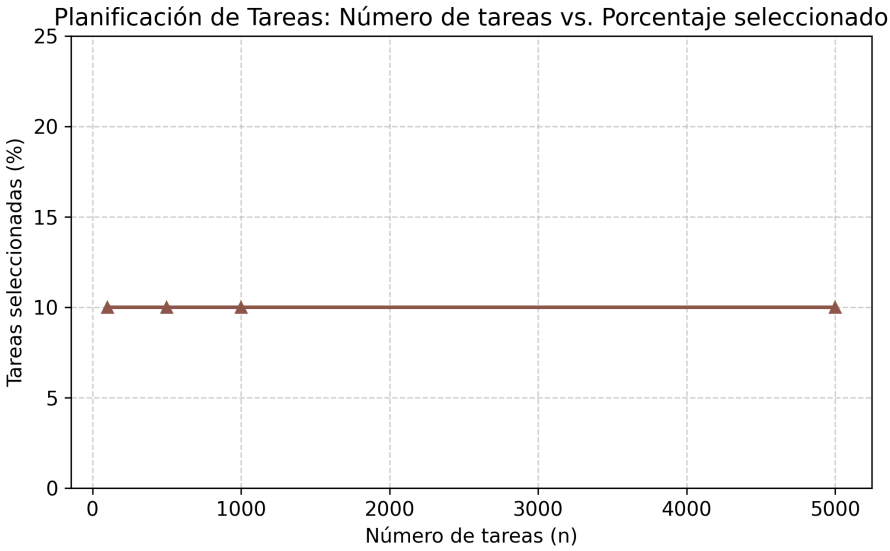
A continuación se recogen las mediciones obtenidas al ejecutar `job_scheduling` para $100, 500, 1000$ y $5000$ tareas con semilla fija ($42$):

| Número de tareas ($n$) | Tiempo medio (s) | Beneficio total | Tareas seleccionadas (%) |
|---|---|---|---|
| 100 | 0.000038 s | 9 578 | 10 (10.00%) |
| 500 | 0.000444 s | 47 584 | 50 (10.00%) |
| 1000 | 0.001290 s | 94 923 | 100 (10.00%) |
| 5000 | 0.024402 s | 473 174 | 500 (10.00%) |

Cuadro 4: Resultados experimentales de `job_scheduling` según el número de tareas.






## 3.2 Cuestiones

1. **¿Cuál es el criterio codicioso utilizado?**
   Ordenar todas las tareas por su beneficio de forma decreciente, intentando ubicar la más valiosa en el último instante libre (slot) disponible que sea menor o igual a su fecha límite (*deadline*).

2. **¿Por qué se procesan primero las tareas con mayor beneficio?**
   Porque el objetivo global del problema es maximizar el beneficio total. Al procesarlas en este orden, garantizamos que las tareas más lucrativas tengan prioridad para quedarse con los recursos limitados de tiempo, dejando que, si hay colisiones, sean las tareas de menor valor las que se queden fuera.

3. **¿Cómo afecta la distribución de deadlines?**
   Si la gran mayoría de tareas presentan *deadlines* muy restrictivos (bajos y próximos entre sí), la cantidad total de tareas que se pueden procesar disminuirá severamente por la alta concentración de colisiones en los primeros instantes de tiempo. Una distribución de deadlines más amplia y uniforme permite ubicar más tareas y en consecuencia, obtener un beneficio mayor.

4. **¿Qué podría suceder si se ordenasen únicamente por deadline?**
   Al priorizar las tareas que expiran antes (sin importar el beneficio), podríamos asignar el tiempo a tareas que expiran rápido pero aportan un beneficio nulo o ínfimo, ocupando espacios valiosos (slots temporales) y desplazando fuera de plazo a tareas de altísimo beneficio que vencían ligeramente más tarde.

5. **¿Cuál es la complejidad temporal y espacial?**
   Ordenar las tareas toma $\mathcal{O}(n \log n)$. Luego, por cada tarea se busca iterativamente de mayor a menor el slot disponible, con un tiempo en el peor de los casos de $\mathcal{O}(D)$, siendo $D$ el *deadline* máximo del sistema. Globalmente la complejidad temporal es $\mathcal{O}(n \log n + n \times D)$. La complejidad espacial es $\mathcal{O}(D)$ para el array que simula el calendario/planificación (`schedule`).

# 4 Cuestiones generales

1. **¿Qué rasgos permiten afirmar que los tres algoritmos estudiados pertenecen al paradigma codicioso?**
   En todos ellos se construye la solución final mediante una secuencia de decisiones *locales* que parecen ser las mejores en el momento (la moneda más grande, la tarea con más beneficio, la actividad que acaba antes). Se elige un elemento, se evalúa su viabilidad, y si es válido se incorpora irrevocablemente al resultado sin poder dar marcha atrás o evaluar caminos alternativos.

2. **¿Qué diferencias existen entre los tres algoritmos en complejidad temporal y espacial?**

   - **Selección de Actividades:** Es el más rápido, con un coste de $\mathcal{O}(n \log n)$ temporal regido sólo por la ordenación, con poco impacto espacial $\mathcal{O}(n)$.
   - **Planificación de tareas:** Introduce dependencia en la escala del tiempo (el deadline máximo $D$). Su coste $\mathcal{O}(n \log n + n \times D)$ lo hace sensible no solo a la cantidad de tareas sino a los plazos permitidos. Espacialmente necesita mantener en memoria el vector del horario $\mathcal{O}(D)$.
   - **Cambio de monedas (Greedy):** Su complejidad está sujeta estrictamente al número de monedas del sistema $m$ ($\mathcal{O}(m \log m)$) y no a la cantidad requerida. Es virtualmente $\mathcal{O}(1)$ espacial, haciéndolo de lejos el más eficiente en memoria e independiente de la magnitud de la entrada global.